

ZERO-TO-JOB-READY DATA ANALYST 2026

M04 - SQL FOR ANALYSTS

Multi-table SQL | cardinality-safe joins | staged analysis | windows

GRAIN -> JOIN -> COUNT -> RECONCILE -> ANALYZE -> DEFEND

Advanced syntax never outranks correct grain, cardinality and validation.

This book is your teacher

- M03 taught single-table SQL. This module introduces every new multi-table/advanced concept before asking you to use it independently.
- Before every JOIN, say the source grains and intended output grain. After every many-side JOIN, count rows before trusting totals.
- Videos make syntax visible. The learner book controls cardinality, reconciliation, deterministic ranking and safe analytical interpretation.
- Protected review remains closed until a genuine attempt is saved. After diagnosis, close the review and solve a changed query from a blank query tab.
- Complex-looking SQL is not the goal. Your goal is a query another analyst can inspect, validate and defend.

Normal M04 loop

1. State business question
2. State source + result grain
3. Watch exact segment if assigned
4. Follow teacher query once
5. Do guided query
6. Close help + changed query
7. Count/reconcile/spot-check
8. Explain cardinality + business meaning

Contents / Index

SA1 INNER JOIN and LEFT JOIN	6
SA2 Cardinality, row multiplication and join validation	9
SA3 String functions for analyst cleaning	12
SA4 Dates and time analysis	15
SA5 Subqueries	18
SA6 CTEs and readable multi-step logic	21
SA7 Window functions and PARTITION BY	24
SA8 Ranking, latest-row logic and LAG/LEAD	27
SA9 UNION, views and performance awareness	30
M-SQLA Integrated analyst SQL project	33
End-of-module mastery checklist	37
Revision / SQL quick reference	38

Module prerequisite and tool map

0. Prerequisite

M03 SQL Foundations competency: SELECT/DISTINCT/WHERE/ORDER BY/LIMIT, aggregates, GROUP BY, HAVING, CASE, NULL handling and result-grain explanation. JOIN, subquery, CTE and window syntax are NOT assumed; M04 teaches them here.

Tools / files

- MySQL Community Server + MySQL Workbench.
- C1_M04_SQL_FOR_ANALYSTS_LAB_SETUP_RC1.sql - controlled multi-table lab setup/reset.
- C1_M04_SQL_FOR_ANALYSTS_PRACTICE_TASKS_RC1.sql - guided + fresh tasks SA1-SA9.
- C1_M04_SQL_FOR_ANALYSTS_EVIDENCE_LOG_RC1.xlsx - append-only evidence/mastery.
- C1_M04_SQL_FOR_ANALYSTS_M_SQLA_PROJECT_RC1.sql - integrated project after SA1-SA9.
- Protected Practice Review and Gate A/B/C reviews - post-attempt only.

Core lab grains

- customers: one row/customer | products: one row/product
- orders: one row/order | order_items: one row/order line
- return_events: one row/return event; an order can have multiple events
- freight_amount belongs to order grain and can be inflated after joining to order_items

Exact lesson, project and Gate route

- SA1 teaches INNER/LEFT JOIN behavior and unmatched rows. Do not jump to SA2 before you can state join output grain.
- SA2 deliberately demonstrates row multiplication and freight inflation. This is the safety foundation for every later multi-table query.
- SA3-SA4 teach analyst cleaning and dates; SA5-SA6 teach nested/staged query structure; SA7-SA8 teach windows/ranking/time comparison; SA9 teaches set stacking and awareness topics.
- After SA1-SA9 fresh evidence, complete M-SQLA and defend the cardinality/reconciliation chain.
- Attempt Gate A independently. A clean pass ends the Gate. Gate B/C require prior Gate failure + protected review/repair + explicit assignment.

Critical-failure examples

Unexplained row multiplication, inflated coarse-grain money, missing one-side uniqueness proof, hidden unmatched records, non-deterministic latest-row logic, broken LAG denominator or failed reconciliation can block mastery even when the query runs.

SA1 - INNER JOIN and LEFT JOIN

Practice block: SA1 in C1_M04_SQL_FOR_ANALYSTS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

Advanced analyst SQL fails most dangerously when a query looks plausible but changes row grain or cardinality invisibly. This lesson teaches one controlled skill and its validation contract.

2. What you are learning

Choose INNER vs LEFT JOIN from the business requirement, state the intended result grain before joining, and prove unmatched behavior with counts.

3. What you already need / prerequisite

M03 SQL Foundations. JOIN syntax is introduced from zero here.

4. Picture it in your head

A join combines rows by keys. INNER keeps matched pairs. LEFT preserves every left-side row and shows NULL on the right when no match exists. Syntax is only the start: you must know what one output row represents.

5. New words before we use them

JOIN = combine rows using a match rule. INNER JOIN = keep matched pairs. LEFT JOIN = keep every left row plus matched right data. Join key = columns used to match. Unmatched = left row with no right match.

6. Watch / visual source

RECOMMENDED. Alex The Analyst - Learn SQL Beginner to Advanced in Under 4 Hours | exact segment 51:10-1:08:03. Why watch: See INNER/LEFT/RIGHT join syntax and output behavior before reproducing the course's grain-first join checks. Internal fallback: Learner book SA1 + practice SQL fully teach INNER vs LEFT, unmatched behavior and output grain.

[Open primary source](#)

SA1 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

Join orders to customers on customer_id. Then LEFT JOIN orders to return_events and verify that orders with no return remain visible with NULL return columns.

8. What you should see / be able to say

State result grain correctly; INNER/LEFT behavior matches the business rule; unmatched count is explicit; no many-side table is treated as one-side without a uniqueness check.

9. Why it worked

The query architecture matches the intended grain, exposes rather than hides cardinality behavior, and leaves an independent validation path. Syntax is accepted only after row counts, keys and important totals behave as expected.

10. Common beginner mistakes

Choosing INNER because it looks simpler and silently dropping unmatched left rows; joining before stating output grain; assuming a key is unique because its name ends in _id; interpreting totals before counting joined rows.

11. If you get stuck - recovery ladder

Unexpected missing rows -> check whether INNER JOIN should be LEFT JOIN. Unexpected extra rows -> inspect key uniqueness and one-to-many relationships before changing filters. Recovery ladder: 1) restate business question; 2) write source/result grain; 3) count keys/rows at the earliest suspicious stage; 4) isolate the smallest query/CTE/window; 5) reveal only a structural hint; 6) open protected review only after the saved attempt.

SA1 - Do alone, validate and master

12. Now you do a small version alone

On a fresh query, join order_items to products for product/category enrichment, then LEFT JOIN a one-row-per-order return summary to orders. Explain why directly joining raw return_events could create extra rows.

13. Validate your result

State result grain correctly; INNER/LEFT behavior matches the business rule; unmatched count is explicit; no many-side table is treated as one-side without a uniqueness check.

14. Teach it back in your own words

Explain INNER vs LEFT JOIN with one business example and state why result grain must be decided before interpreting totals.

15. Protected review + changed fresh retry

Save the SA1 attempt first. If repair is needed, open the protected SA1 review, compare only the weak concept, close it, then solve a changed query from a blank tab. Append the retry in the evidence log; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, source/result grain, key/row-count checks, at least one reconciliation, one failure/recovery note and your own-words explanation. PASS -> next lesson. PARTIAL/FAIL -> targeted repair of this competency only, then changed retry.

SA2 - Cardinality, row multiplication and join validation

Practice block: SA2 in C1_M04_SQL_FOR_ANALYSTS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

Advanced analyst SQL fails most dangerously when a query looks plausible but changes row grain or cardinality invisibly. This lesson teaches one controlled skill and its validation contract.

2. What you are learning

Predict row multiplication, detect one-to-many and many-to-many risk, and reconcile important totals before and after joins.

3. What you already need / prerequisite

Prerequisite: SA1 plus mastered M03 foundations. The new cardinality, row multiplication and join validation idea is still taught explicitly.

4. Picture it in your head

Cardinality describes how many rows on one side can match a row on the other. A correct join can legitimately multiply rows. The danger appears when you sum a measure that belonged to a coarser grain after that multiplication.

5. New words before we use them

Cardinality = how many rows can match across tables. One-to-one = at most one match each side. One-to-many = one parent can match many child rows. Row multiplication = joined output has more rows because matches repeat. Measure grain = row level where a number belongs.

6. Watch / visual source

RECOMMENDED. Alex The Analyst - Learn SQL Beginner to Advanced in Under 4 Hours | exact segment 51:10-1:08:03. Why watch: Reuse the join chapter visually, but focus on the course demonstration that proves row multiplication and repeated order-level freight. Internal fallback: Learner book SA2 is the canonical teacher for cardinality, inflation detection, one-side uniqueness and reconciliation.

[Open primary source](#)

SA2 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

Join orders to order_items. Freight repeats for every item. Compare SUM(freight_amount) from orders alone with the naive post-join sum, then repair the analysis by aggregating item sales to one row/order before joining.

8. What you should see / be able to say

Counts before/after are saved; expected multiplication is explained; right-side uniqueness is checked when assumed; at least one monetary total reconciles at the correct grain.

9. Why it worked

The query architecture matches the intended grain, exposes rather than hides cardinality behavior, and leaves an independent validation path. Syntax is accepted only after row counts, keys and important totals behave as expected.

10. Common beginner mistakes

Adding DISTINCT to hide multiplication; summing an order-level measure after joining to order lines; checking only final money instead of counts/uniqueness; treating legitimate one-to-many expansion as an error without understanding the grain.

11. If you get stuck - recovery ladder

If totals inflate, do not add DISTINCT blindly. Identify the measure grain, aggregate the many side or compute the measure before the multiplication, then reconcile again. Recovery ladder: 1) restate business question; 2) write source/result grain; 3) count keys/rows at the earliest suspicious stage; 4) isolate the smallest query/CTE/window; 5) reveal only a structural hint; 6) open protected review only after the saved attempt.

SA2 - Do alone, validate and master

12. Now you do a small version alone

Create a second validation for return_events: prove how multiple events/order can multiply order rows. Build a one-row-per-order return summary and show the row-count difference.

13. Validate your result

Counts before/after are saved; expected multiplication is explained; right-side uniqueness is checked when assumed; at least one monetary total reconciles at the correct grain.

14. Teach it back in your own words

Explain one-to-one, one-to-many and many-to-many risk using the course tables. Explain why DISTINCT is not a general fix for bad joins.

15. Protected review + changed fresh retry

Save the SA2 attempt first. If repair is needed, open the protected SA2 review, compare only the weak concept, close it, then solve a changed query from a blank tab. Append the retry in the evidence log; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, source/result grain, key/row-count checks, at least one reconciliation, one failure/recovery note and your own-words explanation. PASS -> next lesson. PARTIAL/FAIL -> targeted repair of this competency only, then changed retry.

SA3 - String functions for analyst cleaning

Practice block: SA3 in C1_M04_SQL_FOR_ANALYSTS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

Advanced analyst SQL fails most dangerously when a query looks plausible but changes row grain or cardinality invisibly. This lesson teaches one controlled skill and its validation contract.

2. What you are learning

Use TRIM, UPPER/LOWER, CONCAT and substring logic in query results without overwriting raw values or inventing entity matches.

3. What you already need / prerequisite

Prerequisite: SA2 plus mastered M03 foundations. The new string functions for analyst cleaning idea is still taught explicitly.

4. Picture it in your head

String functions standardize representation, not identity. Trimming spaces or changing case can make comparison/reporting cleaner, but it does not prove that two different spellings represent the same real entity.

5. New words before we use them

TRIM = remove leading/trailing spaces. UPPER/LOWER = change case. CONCAT = join text pieces. Substring = extract part of text. Standardization = make representation consistent; it does not prove two entities are identical.

6. Watch / visual source

RECOMMENDED. Alex The Analyst - Learn SQL Beginner to Advanced in Under 4 Hours | exact segment 1:15:10-1:26:38. Why watch: See common MySQL string functions before applying the safer raw-vs-cleaned analyst pattern. Internal fallback: Learner book SA3 + practice SQL teach TRIM/case/CONCAT without overwriting raw values.

[Open primary source](#)

SA3 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

Return raw customer_name/city beside TRIM(customer_name) and UPPER(TRIM(city)). Add a display label with CONCAT.

8. What you should see / be able to say

Raw and cleaned values remain comparable; only justified presentation changes are made; no UPDATE is used; one potentially ambiguous change is left for investigation.

9. Why it worked

The query architecture matches the intended grain, exposes rather than hides cardinality behavior, and leaves an independent validation path. Syntax is accepted only after row counts, keys and important totals behave as expected.

10. Common beginner mistakes

Updating raw text instead of returning cleaned display fields; using aggressive text rules that change identity; hiding raw values so the transformation cannot be audited; assuming case/space equality proves same customer/entity.

11. If you get stuck - recovery ladder

If cleaning changes business meaning, stop and keep the raw value. If spaces/case still differ, inspect hidden spaces and function order before applying more aggressive logic. Recovery ladder: 1) restate business question; 2) write source/result grain; 3) count keys/rows at the earliest suspicious stage; 4) isolate the smallest query/CTE/window; 5) reveal only a structural hint; 6) open protected review only after the saved attempt.

SA3 - Do alone, validate and master

12. Now you do a small version alone

Build a normalized segment label and a customer display label on a fresh result. Flag any row where raw and cleaned city differ; do not update the source table.

13. Validate your result

Raw and cleaned values remain comparable; only justified presentation changes are made; no UPDATE is used; one potentially ambiguous change is left for investigation.

14. Teach it back in your own words

Explain the difference between standardizing text and deciding that two entities are the same.

15. Protected review + changed fresh retry

Save the SA3 attempt first. If repair is needed, open the protected SA3 review, compare only the weak concept, close it, then solve a changed query from a blank tab. Append the retry in the evidence log; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, source/result grain, key/row-count checks, at least one reconciliation, one failure/recovery note and your own-words explanation. PASS -> next lesson. PARTIAL/FAIL -> targeted repair of this competency only, then changed retry.

SA4 - Dates and time analysis

Practice block: SA4 in C1_M04_SQL_FOR_ANALYSTS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

Advanced analyst SQL fails most dangerously when a query looks plausible but changes row grain or cardinality invisibly. This lesson teaches one controlled skill and its validation contract.

2. What you are learning

Use true date fields with DATEDIFF, YEAR/MONTH and month-level logic while keeping chronological ordering correct.

3. What you already need / prerequisite

Prerequisite: SA3 plus mastered M03 foundations. The new dates and time analysis idea is still taught explicitly.

4. Picture it in your head

Dates are typed values, not just text labels. Derive elapsed days from real dates, and create month-level results using a sortable date/year-month key rather than an alphabetically formatted label alone.

5. New words before we use them

DATE = true date value. DATEDIFF = days between two dates. YEAR/MONTH = date parts. Month key = sortable month representation such as a real month_start date. NULL end date = event not completed/unknown, not zero elapsed days.

6. Watch / visual source

RECOMMENDED_REFERENCE. MySQL 8.0 Reference Manual - Date and Time Functions | DATEDIFF, YEAR, MONTH, DATE/DATE_FORMAT and temporal-value behavior. Why use it: Use the current official MySQL reference for exact date-function behavior and types. Optional Hindi/Urdu visual companion is also preserved. Internal fallback: Learner book SA4 + practice SQL fully teach chronological month keys, DATEDIFF and NULL-safe elapsed-day analysis.

[Open primary source](#)

[Open Hindi/Urdu companion](#)

SA4 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

For completed orders, calculate DATEDIFF(ship_date, order_date), YEAR(order_date), MONTH(order_date), and a month_start expression.

8. What you should see / be able to say

Elapsed days are correct on a manual sample; NULL ship dates are not treated as zero-day shipping; month results sort chronologically; one monthly count reconciles.

9. Why it worked

The query architecture matches the intended grain, exposes rather than hides cardinality behavior, and leaves an independent validation path. Syntax is accepted only after row counts, keys and important totals behave as expected.

10. Common beginner mistakes

Formatting text as YYYY-MM and then sorting alphabetically across years; treating NULL ship dates as zero days; calculating elapsed days from strings without checking types; forgetting to validate one month directly.

11. If you get stuck - recovery ladder

If date functions return NULL/errors, verify data type and missing ship_date before changing formulas. If months sort incorrectly, use a true month_start/year-month key. Recovery ladder: 1) restate business question; 2) write source/result grain; 3) count keys/rows at the earliest suspicious stage; 4) isolate the smallest query/CTE/window; 5) reveal only a structural hint; 6) open protected review only after the saved attempt.

SA4 - Do alone, validate and master

12. Now you do a small version alone

Build a monthly completed-order summary and a late-shipment rate. Validate one month by filtering the source orders directly.

13. Validate your result

Elapsed days are correct on a manual sample; NULL ship dates are not treated as zero-day shipping; month results sort chronologically; one monthly count reconciles.

14. Teach it back in your own words

Explain why DATE_FORMAT text can be useful for display but a true date/month key is safer for chronological analysis.

15. Protected review + changed fresh retry

Save the SA4 attempt first. If repair is needed, open the protected SA4 review, compare only the weak concept, close it, then solve a changed query from a blank tab. Append the retry in the evidence log; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, source/result grain, key/row-count checks, at least one reconciliation, one failure/recovery note and your own-words explanation. PASS -> next lesson. PARTIAL/FAIL -> targeted repair of this competency only, then changed retry.

SA5 - Subqueries

Practice block: SA5 in C1_M04_SQL_FOR_ANALYSTS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

Advanced analyst SQL fails most dangerously when a query looks plausible but changes row grain or cardinality invisibly. This lesson teaches one controlled skill and its validation contract.

2. What you are learning

Use scalar/list/table-like subqueries when they make the question clearer, and explain the scope of the inner and outer query.

3. What you already need / prerequisite

Prerequisite: SA4 plus mastered M03 foundations. The new subqueries idea is still taught explicitly.

4. Picture it in your head

A subquery produces a result used by another query. The important question is what shape it returns: one value, a list, or a table-like set. Prefer the clearest correct structure rather than nesting for style.

5. New words before we use them

Subquery = query used by another query. Scalar = exactly one value. List/set = multiple values. Inner query = nested query producing input. Outer query = query consuming that result. Result shape/grain determines how nesting can be used safely.

6. Watch / visual source

RECOMMENDED. Alex The Analyst - Learn SQL Beginner to Advanced in Under 4 Hours | exact segment 1:35:16-1:45:57. Why watch: See scalar and nested subquery patterns before reasoning about inner-result shape. Internal fallback: Learner book SA5 + practice SQL teach scalar/list/table-like result shape and validation.

[Open primary source](#)

SA5 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

Return order_items above average line_sales using a scalar subquery. Then compute customers above average customer sales with an aggregated inner result.

8. What you should see / be able to say

Inner result shape matches how the outer query uses it; thresholds are independently queryable; result contains only rows/groups that satisfy the comparison.

9. Why it worked

The query architecture matches the intended grain, exposes rather than hides cardinality behavior, and leaves an independent validation path. Syntax is accepted only after row counts, keys and important totals behave as expected.

10. Common beginner mistakes

Expecting a scalar but inner query returns multiple rows; nesting merely to look advanced; validating only outer output; using a subquery where a clearer staged CTE would be easier to inspect.

11. If you get stuck - recovery ladder

Subquery returns more than one row where one scalar is expected -> inspect the inner grain. Nested query feels unreadable -> rewrite with a CTE and compare clarity. Recovery ladder: 1) restate business question; 2) write source/result grain; 3) count keys/rows at the earliest suspicious stage; 4) isolate the smallest query/CTE/window; 5) reveal only a structural hint; 6) open protected review only after the saved attempt.

SA5 - Do alone, validate and master

12. Now you do a small version alone

Return products whose total line_sales exceed the average product total. Use a subquery and validate the threshold separately.

13. Validate your result

Inner result shape matches how the outer query uses it; thresholds are independently queryable; result contains only rows/groups that satisfy the comparison.

14. Teach it back in your own words

Explain scalar vs list/table-like subquery roles with a simple example.

15. Protected review + changed fresh retry

Save the SA5 attempt first. If repair is needed, open the protected SA5 review, compare only the weak concept, close it, then solve a changed query from a blank tab. Append the retry in the evidence log; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, source/result grain, key/row-count checks, at least one reconciliation, one failure/recovery note and your own-words explanation. PASS -> next lesson. PARTIAL/FAIL -> targeted repair of this competency only, then changed retry.

SA6 - CTEs and readable multi-step logic

Practice block: SA6 in C1_M04_SQL_FOR_ANALYSTS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

Advanced analyst SQL fails most dangerously when a query looks plausible but changes row grain or cardinality invisibly. This lesson teaches one controlled skill and its validation contract.

2. What you are learning

Use WITH to name intermediate query results, preserve deliberate grains between stages, and validate each stage independently.

3. What you already need / prerequisite

Prerequisite: SA5 plus mastered M03 foundations. The new ctes and readable multi-step logic idea is still taught explicitly.

4. Picture it in your head

A CTE is a named query result that lives for one statement. It is useful for multi-step reasoning because each stage can have an explicit grain and validation check. It is not a permanently stored table.

5. New words before we use them

CTE = Common Table Expression created with WITH for one statement. Stage = named intermediate result. Stage grain = what one row represents at that step. Reconciliation = independent check that important totals/counts agree.

6. Watch / visual source

RECOMMENDED. Alex The Analyst - Learn SQL Beginner to Advanced in Under 4 Hours | exact segment 1:58:55-2:09:06. Why watch: See WITH/CTE syntax, then use the course's grain-at-each-stage method rather than writing one opaque query. Internal fallback: Learner book SA6 + practice SQL teach staged grain and independent CTE validation.

[Open primary source](#)

SA6 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

Build order_sales as one row/order, validate count/total, then customer_sales as one row/customer and return the top customers.

8. What you should see / be able to say

Each CTE has a stated grain; stage counts are plausible; aggregate totals reconcile to the source; final output uses the intended stage rather than raw many-side rows.

9. Why it worked

The query architecture matches the intended grain, exposes rather than hides cardinality behavior, and leaves an independent validation path. Syntax is accepted only after row counts, keys and important totals behave as expected.

10. Common beginner mistakes

Writing one huge CTE chain without validating stages; naming a CTE vaguely so grain is hidden; changing grain accidentally between stages; assuming CTE automatically improves performance.

11. If you get stuck - recovery ladder

If a later CTE has surprising rows, run the previous stage alone. If a CTE name hides its grain, rename it to make the row unit obvious. Recovery ladder: 1) restate business question; 2) write source/result grain; 3) count keys/rows at the earliest suspicious stage; 4) isolate the smallest query/CTE/window; 5) reveal only a structural hint; 6) open protected review only after the saved attempt.

SA6 - Do alone, validate and master

12. Now you do a small version alone

Create a `monthly_sales` CTE and a second CTE for regional monthly sales. Use the final query to compare region share by month and reconcile each month to the monthly total.

13. Validate your result

Each CTE has a stated grain; stage counts are plausible; aggregate totals reconcile to the source; final output uses the intended stage rather than raw many-side rows.

14. Teach it back in your own words

Explain why CTEs improve reasoning/validation even when the same query could be written as one nested statement.

15. Protected review + changed fresh retry

Save the SA6 attempt first. If repair is needed, open the protected SA6 review, compare only the weak concept, close it, then solve a changed query from a blank tab. Append the retry in the evidence log; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, source/result grain, key/row-count checks, at least one reconciliation, one failure/recovery note and your own-words explanation. PASS -> next lesson. PARTIAL/FAIL -> targeted repair of this competency only, then changed retry.

SA7 - Window functions and PARTITION BY

Practice block: SA7 in C1_M04_SQL_FOR_ANALYSTS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

Advanced analyst SQL fails most dangerously when a query looks plausible but changes row grain or cardinality invisibly. This lesson teaches one controlled skill and its validation contract.

2. What you are learning

Use aggregate windows to add group context while preserving detail rows, and distinguish window output grain from GROUP BY output grain.

3. What you already need / prerequisite

Prerequisite: SA6 plus mastered M03 foundations. The new window functions and partition by idea is still taught explicitly.

4. Picture it in your head

GROUP BY collapses detail to one row/group. A window calculation can keep detail rows and attach a group-level measure to each row. PARTITION BY defines the peer group; ORDER BY inside the window can define sequence.

5. New words before we use them

Window function = calculation across related rows without necessarily collapsing them. OVER = defines the window. PARTITION BY = peer group. Window order = sequence inside the calculation. Detail grain = original row-level detail retained.

6. Watch / visual source

RECOMMENDED. Alex The Analyst - Learn SQL Beginner to Advanced in Under 4 Hours | exact segment 1:45:58-1:58:55. Why watch: See OVER and PARTITION BY before comparing a window result with GROUP BY on the same data. Internal fallback: Learner book SA7 + practice SQL teach detail-grain preservation and partition validation.

[Open primary source](#)

SA7 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

On one row/order, add customer order count and customer freight total with COUNT/SUM OVER(PARTITION BY customer_id).

8. What you should see / be able to say

Window query preserves the intended detail row count; partition totals match a separate GROUP BY check; ordering is deterministic where sequence matters.

9. Why it worked

The query architecture matches the intended grain, exposes rather than hides cardinality behavior, and leaves an independent validation path. Syntax is accepted only after row counts, keys and important totals behave as expected.

10. Common beginner mistakes

Using GROUP BY and losing detail when the goal is row-level output; confusing PARTITION BY with result sorting; failing to validate partition totals against GROUP BY; using non-deterministic order for cumulative logic.

11. If you get stuck - recovery ladder

If row count collapses, you probably used GROUP BY instead of a window. If running totals are unstable on ties, add a deterministic tie-breaker to ORDER BY. Recovery ladder: 1) restate business question; 2) write source/result grain; 3) count keys/rows at the earliest suspicious stage; 4) isolate the smallest query/CTE/window; 5) reveal only a structural hint; 6) open protected review only after the saved attempt.

SA7 - Do alone, validate and master

12. Now you do a small version alone

Create one row/order with customer cumulative order sales using an order-level sales CTE plus SUM(...) OVER(PARTITION BY customer_id ORDER BY order_date, order_id).

13. Validate your result

Window query preserves the intended detail row count; partition totals match a separate GROUP BY check; ordering is deterministic where sequence matters.

14. Teach it back in your own words

Explain PARTITION BY and why a window can show customer totals without reducing to one row/customer.

15. Protected review + changed fresh retry

Save the SA7 attempt first. If repair is needed, open the protected SA7 review, compare only the weak concept, close it, then solve a changed query from a blank tab. Append the retry in the evidence log; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, source/result grain, key/row-count checks, at least one reconciliation, one failure/recovery note and your own-words explanation. PASS -> next lesson. PARTIAL/FAIL -> targeted repair of this competency only, then changed retry.

SA8 - Ranking, latest-row logic and LAG/LEAD

Practice block: SA8 in C1_M04_SQL_FOR_ANALYSTS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

Advanced analyst SQL fails most dangerously when a query looks plausible but changes row grain or cardinality invisibly. This lesson teaches one controlled skill and its validation contract.

2. What you are learning

Use ROW_NUMBER for deterministic latest-row selection, RANK/DENSE_RANK for ties, and LAG/LEAD for prior/next-period comparisons.

3. What you already need / prerequisite

Prerequisite: SA7 plus mastered M03 foundations. The new ranking, latest-row logic and lag/lead idea is still taught explicitly.

4. Picture it in your head

Ranking/window functions depend on explicit partition and ordering. ROW_NUMBER chooses one deterministic row only if the ORDER BY fully resolves ties. LAG/LEAD reaches adjacent rows according to the window order; it does not calculate a growth percentage automatically.

5. New words before we use them

ROW_NUMBER = unique sequential rank. RANK = ties share rank and leave gaps. DENSE_RANK = ties share rank without gaps. LAG/LEAD = prior/next row value in window order. Tie-breaker = extra ORDER BY field that makes selection deterministic.

6. Watch / visual source

RECOMMENDED. Alex The Analyst - Learn SQL Beginner to Advanced in Under 4 Hours | exact segment 1:45:58-1:58:55. Why watch: Use the window chapter for ranking/LAG syntax, then follow the course's deterministic tie-breaker and safe denominator rules. Internal fallback: Learner book SA8 + practice SQL fully teach ROW_NUMBER/RANK/DENSE_RANK/LAG/LEAD behavior.

[Open primary source](#)

SA8 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

Rank orders per customer by order_date DESC, order_id DESC and keep rn=1. Build monthly sales and use LAG(monthly_sales) for prior-month comparison.

8. What you should see / be able to say

Latest/second-latest logic has deterministic tie-breaker; row count is one/customer where expected; monthly LAG aligns to chronological month_start; percentage change handles NULL/zero safely.

9. Why it worked

The query architecture matches the intended grain, exposes rather than hides cardinality behavior, and leaves an independent validation path. Syntax is accepted only after row counts, keys and important totals behave as expected.

10. Common beginner mistakes

ROW_NUMBER latest logic without full tie-breaker; confusing RANK and DENSE_RANK gaps; calculating percent change when prior value is zero/NULL; LAG ordered by month label instead of chronological key.

11. If you get stuck - recovery ladder

Unexpected duplicate latest rows -> ROW_NUMBER/tie-breaker issue. Wrong previous month -> verify month sequence and missing-month behavior before changing arithmetic. Recovery ladder: 1) restate business question; 2) write source/result grain; 3) count keys/rows at the earliest suspicious stage; 4) isolate the smallest query/CTE/window; 5) reveal only a structural hint; 6) open protected review only after the saved attempt.

SA8 - Do alone, validate and master

12. Now you do a small version alone

Find each customer's second-most-recent order and build month-over-month absolute + percentage sales change with a safe zero/NULL denominator rule.

13. Validate your result

Latest/second-latest logic has deterministic tie-breaker; row count is one/customer where expected; monthly LAG aligns to chronological month_start; percentage change handles NULL/zero safely.

14. Teach it back in your own words

Explain the difference between ROW_NUMBER, RANK and DENSE_RANK, then explain what LAG returns.

15. Protected review + changed fresh retry

Save the SA8 attempt first. If repair is needed, open the protected SA8 review, compare only the weak concept, close it, then solve a changed query from a blank tab. Append the retry in the evidence log; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, source/result grain, key/row-count checks, at least one reconciliation, one failure/recovery note and your own-words explanation. PASS -> next lesson. PARTIAL/FAIL -> targeted repair of this competency only, then changed retry.

SA9 - UNION, views and performance awareness

Practice block: SA9 in C1_M04_SQL_FOR_ANALYSTS_PRACTICE_TASKS_RC1.sql

1. Why this lesson matters

Advanced analyst SQL fails most dangerously when a query looks plausible but changes row grain or cardinality invisibly. This lesson teaches one controlled skill and its validation contract.

2. What you are learning

Use UNION ALL/UNION correctly for compatible result sets and explain views, temporary results and indexes at analyst-awareness depth.

3. What you already need / prerequisite

Prerequisite: SA8 plus mastered M03 foundations. The new union, views and performance awareness idea is still taught explicitly.

4. Picture it in your head

UNION ALL stacks compatible rows and keeps duplicates. UNION additionally deduplicates selected rows, which can hide meaningful repetition. A view stores a query definition; indexes can speed some lookups but are a database-design/operations topic beyond this module's required depth.

5. New words before we use them

UNION ALL = stack compatible result sets and retain duplicates. UNION = stack then deduplicate selected rows. View = stored query definition. Temporary table/result = short-lived materialized intermediate data. Index = database structure that can speed some access patterns.

6. Watch / visual source

RECOMMENDED. Alex The Analyst - Learn SQL Beginner to Advanced in Under 4 Hours | exact segment 1:08:04-1:15:09. Why watch: See UNION vs UNION ALL before deciding whether duplicate rows are meaningful or should be removed. Internal fallback: Learner book SA9 + practice SQL teach UNION semantics and keep views/indexes at analyst-awareness depth.

[Open primary source](#)

SA9 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

Stack two compatible customer_id result sets with UNION ALL, then UNION, and identify a customer that appears twice only with UNION ALL.

8. What you should see / be able to say

SELECT lists are type/column compatible; UNION ALL keeps meaningful duplicates; UNION use is justified rather than automatic; no server-administration task is performed.

9. Why it worked

The query architecture matches the intended grain, exposes rather than hides cardinality behavior, and leaves an independent validation path. Syntax is accepted only after row counts, keys and important totals behave as expected.

10. Common beginner mistakes

Using UNION when meaningful duplicate cohort membership should remain; mismatched column order/types; assuming a view stores a frozen copy of data; turning performance awareness into unsupported index/server administration.

11. If you get stuck - recovery ladder

UNION column-count/type error -> align SELECT lists. Missing repeated business facts -> check whether UNION accidentally removed duplicates that should remain. Recovery ladder: 1) restate business question; 2) write source/result grain; 3) count keys/rows at the earliest suspicious stage; 4) isolate the smallest query/CTE/window; 5) reveal only a structural hint; 6) open protected review only after the saved attempt.

SA9 - Do alone, validate and master

12. Now you do a small version alone

Create two compatible result sets labeling customers as HighSales and RecentBuyer, combine with UNION ALL, and explain why one customer may legitimately have two rows with different labels.

13. Validate your result

SELECT lists are type/column compatible; UNION ALL keeps meaningful duplicates; UNION use is justified rather than automatic; no server-administration task is performed.

14. Teach it back in your own words

Explain UNION vs UNION ALL and give one sentence each for view, temporary table and index at analyst awareness level.

15. Protected review + changed fresh retry

Save the SA9 attempt first. If repair is needed, open the protected SA9 review, compare only the weak concept, close it, then solve a changed query from a blank tab. Append the retry in the evidence log; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Save runnable SQL, source/result grain, key/row-count checks, at least one reconciliation, one failure/recovery note and your own-words explanation. PASS -> next lesson. PARTIAL/FAIL -> targeted repair of this competency only, then changed retry.

M-SQLA - Integrated analyst SQL project

1. Why this project matters

Employers need proof that you can combine multi-table SQL without losing grain, inflating measures or hiding unmatched data. This project makes that reasoning visible and defensible.

2. What you are building

A multi-table management analysis explaining customer/product sales drivers, monthly change, shipping delays and return patterns with explicit grain, cardinality, staged SQL and independent reconciliations.

3. Prerequisite

SA1-SA9 fresh evidence. Do not use a join/subquery/CTE/window/rank pattern in this project unless its lesson has already been completed.

4. Picture it in your head

Raw tables at different grains -> key/cardinality proof -> one-row/order sales stage -> safe enrichment -> month/customer/product stages -> windows/ranking -> reconciliations -> evidence-supported findings. Trace errors backward to the first wrong grain.

5. New words before we use them

Driver = segment/entity associated with more of the observed metric. Cohort = labelled group. Reconciliation = independent total/count agreement. Observation = what data shows. Hypothesis = possible explanation not yet proven. Query architecture = deliberate stages and grains.

6. Watch / visual source

INTERNAL PROJECT TEACHER. No new external prerequisite is added. Rewatch only the lesson-specific visual for a named weak competency; project mastery must come from your SQL and validations.

M-SQLA - Beginner setup and worked micro-example

7. Follow with me once - fully worked teacher demonstration

Write each source grain. Prove customers.customer_id and products.product_id are unique. Build order_sales as one row/order_id by summing order_items.line_sales. Validate COUNT(order_sales) against orders and SUM(order_sales) against raw line_sales. Then join that one-row/order stage to orders. Compare this safe route with a naive orders x order_items join that repeats freight_amount.

8. What you should see / be able to say

You can show exactly where line-grain sales become order-grain sales, why freight stays one value/order, and which row-count/total checks prove no accidental multiplication corrupted the result.

9. Why it worked

The micro-example resolves the most dangerous problem - mixed grains - before the project expands. Later customer/month/product analysis inherits a validated order-level base instead of multiplying coarse-grain measures.

10. Common beginner mistakes

Starting from a giant five-table join; trusting DISTINCT; adding freight after joining line rows; joining raw return_events directly to line-grain sales; hiding unknown keys; using window rankings before building the correct base grain.

11. If you get stuck - recovery ladder

Stop the final report. Run each stage separately. Write its row grain beside the CTE name. Count rows and sum the owned measure. If a join multiplies rows, determine whether that expansion is expected before continuing.

M-SQLA - Full independent build and validation

12. Now you do the project

- Preserve the five lab tables; do not UPDATE/DELETE source fixtures.
- Write business question, source grains, final result grains and measure ownership before querying.
- Build and validate one-row/order order_sales; then enrich to customer/product/date safely.
- Use string/date logic only where justified and keep raw descriptive fields available for comparison.
- Build customer and product driver summaries with share/rank; build monthly sales and LAG absolute/% change.
- Build late-shipment analysis from completed orders with true dates and a stated late rule.
- Aggregate return_events to one row/order before joining to order-level sales; compare groups descriptively, without claiming causation.
- Use a window aggregate at detail grain, deterministic ROW_NUMBER latest-row logic, and RANK/DENSE_RANK with tie behavior explained.
- Create two compatible labelled cohorts and combine with UNION ALL; justify whether duplicate membership should remain.
- Save source counts, one-side uniqueness, unmatched counts, join counts, raw-vs-stage line sales, raw-vs-corrected freight, monthly reconciliation and one customer/product spot-check.

13. Validate before interpreting

At minimum: raw line_sales total = order-level CTE total; raw freight total = corrected order-grain freight total; one monthly total reconciles independently; every assumed one-side key is proven unique; unmatched keys/events are counted rather than silently removed.

M-SQLA - Explain, evidence and project rubric

14. Teach it back in your own words

Give a 2-minute defense: business question -> source grains -> dangerous cardinality -> safe staged architecture -> key validations -> strongest finding -> one hypothesis you did not claim as fact -> limitation -> recommendation.

15. Protected review + changed fresh retry

Save the complete project attempt first. If one competency is weak, open only the matching lesson/project review. Close answer material and rebuild the affected stage on a changed question/cohort before mastery.

16. Evidence / mastery + exact next route

Critical pass requirements: source grains explicit; one-side keys checked; row multiplication explained; coarse-grain measures protected; unmatched records visible; staged query grains readable; windows/ranking deterministic; LAG percent safe for NULL/zero; at least two independent reconciliations; findings supported; limitations stated; learner can defend the architecture without review. A polished SQL script cannot compensate for inflated totals. PASS -> Gate A. PARTIAL/FAIL -> targeted repair and changed project evidence before Gate A.

End-of-module mastery checklist

- SA1: Choose INNER vs LEFT from the business requirement and prove unmatched behavior.
- SA2: Predict/detect row multiplication; protect coarse-grain measures; reconcile before/after joins.
- SA3: Return auditable cleaned string fields without overwriting raw identity.
- SA4: Use true dates, elapsed days and chronological monthly analysis with NULL-safe rules.
- SA5: Use subqueries only with understood inner-result shape/grain and independent threshold validation.
- SA6: Build readable CTE stages with explicit grain and stage-level counts/totals.
- SA7: Use windows to add group context while preserving intended detail rows; reconcile partitions.
- SA8: Use deterministic latest/rank logic and safe LAG/LEAD period comparison.
- SA9: Use UNION ALL/UNION intentionally and explain view/temp/index concepts at analyst depth.
- M-SQLA: Defend a multi-table analysis whose money, row counts and interpretations reconcile.

Gate readiness

Attempt Gate A only when cardinality-safe joins and the M-SQLA reconciliations are fresh evidence.

A clean Gate A pass ends M04. B/C are remediation full retests only.

Revision / SQL quick reference

INNER vs LEFT	INNER keeps matches; LEFT preserves all left rows and shows NULL for unmatched right.
Cardinality	Count rows + prove one-side uniqueness before trusting post-join totals.
Measure ownership	Know the grain a measure belongs to before joining to finer-grain tables.
TRIM/case	Standardize representation in results; do not invent identity equivalence.
Dates	True chronological key + NULL-safe elapsed-day logic.
Subquery	Know inner result shape: scalar/list/table-like.
CTE	Name a stage and its grain; validate stages separately.
Window	Usually preserve detail; PARTITION BY defines peer group.
ROW_NUMBER	Latest row needs full deterministic ORDER BY tie-breaker.
LAG	Returns prior ordered row value; percent change needs NULL/zero denominator rule.
UNION ALL	Stacks and keeps rows; UNION additionally deduplicates selected rows.

Next route: M-SQLA -> Gate A -> M05 Power BI Modeling only after SQL for Analysts competency pass.